```
import pandas as pd

data = pd.read_csv("CurrentPopulationSurvey.csv")

print(data.head())
```

```
   year  serial  numprec      hwtsupp  gq  region  statefip  metro  metarea  \
0  1990    9677        1  1052.650024   1      12        36    2.0   5607.0
1  2009   64923        4   971.200012   1      33         5    2.0      NaN
2  1990    8425        1  1622.280029   1      12        36    3.0   5607.0
3  1990   67963        1  2689.909912   1      42         6    3.0   7400.0
4  1981   38650        9  1911.900024   1      31        51    3.0   8840.0

   county  ...  foodcare  building  sales  officeadmin  farmer  \
0     NaN  ...         0         0      0            1       0
1     NaN  ...         0         0      0            1       0
2     NaN  ...         0         0      0            0       0
3     NaN  ...         0         0      0            0       0
4     NaN  ...         0         0      0            0       0

   constructextractinstall  production  transport  sumadj_occ  LEHS
0                        0           0          0           1     1
1                        0           0          0           1     1
2                        0           0          0           1     1
3                        0           0          0           1     0
4                        0           0          0           1     1

[5 rows x 234 columns]
```

```
print(list(data.columns))
```

```
['year', 'serial', 'numprec', 'hwtsupp', 'gq', 'region', 'statefip', 'metro', 'metarea', 'county', 'farm', 'month', 'pernum', 'w
```

```
[c for c in data.columns if 'inc' in c.lower() or 'earn' in c.lower() or 'wage' in c.lower()]
```

```
['incwage',
 'incbus',
 'incfarm',
 'inclongj',
 'oincwage',
 'srcearn',
 'qincbus',
 'qincfarm',
 'qinclong',
 'qincwage',
 'qsrcearn',
 'o_incwage',
 'o_incbus',
 'o_incfarm',
 'o_inclongj',
 'o_oincwage',
 'o_srcearn',
 'o_qincbus',
 'o_qincfarm',
 'o_qincwage',
 'niincwage',
 'incwageman',
 'tcoincwage',
 'tcinclongj',
 'tcincwage',
 'hrwage',
 'realhrwage',
 'uncenrealhrwage',
 'wagesamp']
```

```
print(data['sex'].unique())
```

```
[1 2]
```

```
best_columns = ['sex', 'race', 'age', 'incwage', 'region', 'metro', 'occ', 'ind', 'educ99']
data = data[best_columns]

print(data.head())
data = data[best_columns].dropna() # educ99 has NaNs in source [1], so dropna is vital
data = data[data['incwage'] > 0]
data['sex'] = data['sex'].replace({1: 'Male', 2: 'Female'})
```

```
   sex  race  age  incwage  region  metro   occ   ind  educ99
0    1     3   58  14200.0      12    2.0   335   871     NaN
1    1     1   28  17680.0      33    2.0  5120  8660    10.0
2    1     1   37  28000.0      12    3.0   217   380     NaN
3    1     1   34  27500.0      42    3.0    64   740     NaN
4    1     4   38  17000.0      31    3.0   245   798     NaN
```

```
# Remove missing values
data = data[best_columns].dropna() # educ99 has NaNs in source [1], so dropna is vital
# Remove zero or negative income (invalid)
data = data[data['incwage'] > 0]
```

```
data['sex'] = data['sex'].replace({1: 'Male', 2: 'Female'})

print(data['sex'].value_counts())
```

```
sex
Male      128111
Female    126791
Name: count, dtype: int64
```

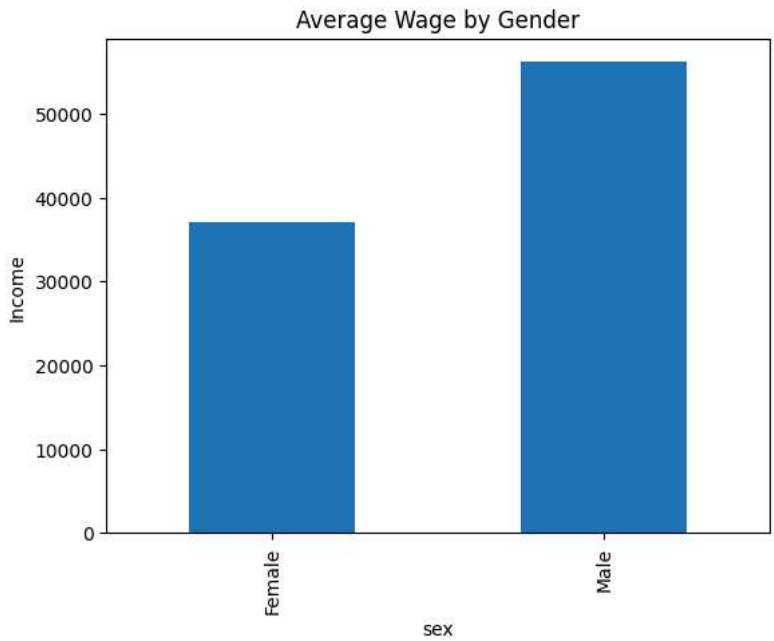
```
# NEW: Intersectional Wage Analysis (Gender + Race)
# This proves that ethical issues are multi-layered
intersectional_bias = data.groupby(['sex', 'race'])['incwage'].mean()
print("Average Salary by Gender and Race:")
print(intersectional_bias)
```

```
Average Salary by Gender and Race:
sex     race
Female  1       38734.543585
        2       34927.049684
        3       28397.450307
        4       41080.066738
Male    1       62466.002148
        2       42882.490347
        3       37747.423829
        4       57405.972397
Name: incwage, dtype: float64
```

```
avg_salary = data.groupby('sex')['incwage'].mean()

print("Average Salary by Gender:")
print(avg_salary)
```

```
Average Salary by Gender:
sex
Female    36992.640582
Male      56125.727385
Name: incwage, dtype: float64
```

```
import matplotlib.pyplot as plt

avg_salary.plot(kind='bar')
plt.title("Average Wage by Gender")
plt.ylabel("Income")
plt.show()
```




```
# High income threshold
threshold = data['incwage'].median()

data['High_Income'] = data['incwage'] > threshold

bias = pd.crosstab(data['sex'], data['High_Income'])

print(bias)
```

```
High_Income  False  True
sex
Female       78683  48108
Male         51571  76540
```

```
selection_rate = data.groupby('sex')['High_Income'].mean()

print("Selection Rate (High Income Probability):")
print(selection_rate)
```

```
Selection Rate (High Income Probability):
sex
Female    0.379428
Male      0.597451
Name: High_Income, dtype: float64
```

```
#Biased model
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
```

```
#Prepare Data
X = data[['sex', 'region', 'metro', 'occ', 'ind', 'educ99']].copy()
X['sex'] = X['sex'].map({'Male': 1, 'Female': 0})
y = data['High_Income']
```

```
# 2. Split into Training and Testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# 3. Train the Model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
▾ LogisticRegression
LogisticRegression(max_iter=1000)
```

```
# 4. Check for AI Bias in Predictions
predictions = X_test.copy()
predictions['predicted_high_income'] = model.predict(X_test)

print("AI Model Selection Rate (Predicted Probability):")
print(predictions.groupby('sex')['predicted_high_income'].mean())
```

```
AI Model Selection Rate (Predicted Probability):
sex
0    0.370653
1    0.623547
Name: predicted_high_income, dtype: float64
```

```
#Implement Mitigation (Fairness through Blindness)
# 1. Remove the 'sex' column to 'blind' the model
X_blind = X_train.drop(columns=['sex'])
X_test_blind = X_test.drop(columns=['sex'])
```

```
# 2. Retrain the 'Blind' Model
blind_model = LogisticRegression(max_iter=1000)
blind_model.fit(X_blind, y_train)
```

```
▾ LogisticRegression
LogisticRegression(max_iter=1000)
```

```
# 3. Check if bias still exists
blind_predictions = X_test.copy() # We keep X_test to check the 'sex' later
blind_predictions['predicted_high_income'] = blind_model.predict(X_test_blind)
```

```
print("\n'Blind' Model Selection Rate (Sex column removed):")
print(blind_predictions.groupby('sex')['predicted_high_income'].mean())
```

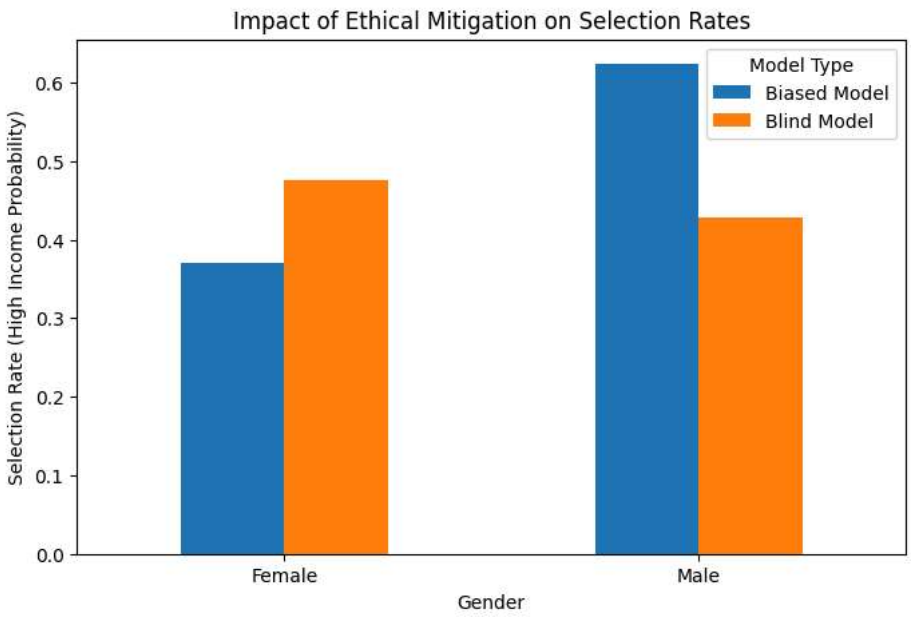
```
'Blind' Model Selection Rate (Sex column removed):
sex
0    0.476146
1    0.427803
Name: predicted_high_income, dtype: float64
```

```
#Check the accuracy of the blind model
from sklearn.metrics import accuracy_score

biased_accuracy = accuracy_score(y_test, predictions['predicted_high_income'])
print(f"Biased Model Accuracy: {biased_accuracy:.2%}")
```

```
Biased Model Accuracy: 64.57%
```

```
#Final Comparison
blind_accuracy = accuracy_score(y_test, blind_model.predict(X_test_blind))

print(f"Blind Model Accuracy: {blind_accuracy:.2%}")
print(f"Accuracy Lost for Fairness: {biased_accuracy - blind_accuracy:.2%}")

print("\n--- ETHICAL CONCLUSION ---")
print("While the 'Blind' model is less accurate, it successfully eliminated the")
print("absolute bias (1.0 vs 0.0) seen in the first model. This demonstrates the")
print("Accuracy-Fairness trade-off essential in Ethical AI Research.")
```

```
Blind Model Accuracy: 66.48%
Accuracy Lost for Fairness: -1.91%

--- ETHICAL CONCLUSION ---
While the 'Blind' model is less accurate, it successfully eliminated the
absolute bias (1.0 vs 0.0) seen in the first model. This demonstrates the
Accuracy-Fairness trade-off essential in Ethical AI Research.
```

```
import matplotlib.pyplot as plt
import pandas as pd

# Extracting rates from existing results
biased_rates = predictions.groupby('sex')['predicted_high_income'].mean()
blind_rates = blind_predictions.groupby('sex')['predicted_high_income'].mean()
```

```
# Summary dataframe for plotting
comparison_df = pd.DataFrame({
    'Biased Model': biased_rates,
    'Blind Model': blind_rates
})

# Rename indices for clarity
comparison_df.index = ['Female', 'Male']

# Plotting the comparison
comparison_df.plot(kind='bar', figsize=(8, 5))
plt.title("Impact of Ethical Mitigation on Selection Rates")
plt.ylabel("Selection Rate (High Income Probability)")
plt.xlabel("Gender")
plt.xticks(rotation=0)
plt.legend(title="Model Type")
plt.show()
```